Tutorial 1 - Project build automation with Gradle

SENG202 - Software Engineering Project Workshop

SENG202 Teaching Team

21st July 2026

1 Introduction

This tutorial will walk you through setting up a project using Gradle. Gradle is a build automation and dependency management utility which is required for this course. all of your should have experience with Gradle from SENG201 It is popular in industry and understanding how to use it (and other similar tools) will be very useful.

Each student should complete this individually. We will begin with some background information on Gradle (mainly based on Gradle's official documentation). **If you skip this part, do not be surprised if the remainder of this tutorial leaves you feeling confused.**

1.1 Objectives

- Understand what Gradle is and the problems it can solve;
- Make sure Gradle is installed on your development machines;
- Become comfortable with setting up a project with Gradle;
- Learn how to import a Gradle project into an IDE (IntelliJ);
- Understand how dependency management in Gradle works;
- Use Gradle to build a project from source (produce an executable);
- Complete the Gradle quiz on the Quiz Server. The quiz will be open in the second half of the tutorial time, and close at the end of the tutorial.

2 Gradle and the Build Lifecycle

Gradle began as an attempt to support the build processes of big and complex systems, usually compound of several projects, each of them with their own build files. The aim was to have a standard way to build projects, a clear definition of what the project consisted of an easy way to publish project information, and a way to share executable files across several projects. Using the settings file (build.gradle), Gradle will manage your projects build configurations so you can easily build, test, report and document your projects source code.

3 The Build Lifecycle

The source code commonly written by software engineers is not very useful by itself. It must be compiled into executable binary instructions and linked to any dependencies (e.g., libraries) which the programmer has imported. There are also many other jobs that professional engineers need to perform when they want to convert their work into a real executable program. All these jobs (including compilation, linking, automated testing, and others) form what is known as the "build process".

Gradle is a build automation tool, which means that it carries out these tasks automatically (software engineers love writing tools to do their work for them). Gradle is based around the central concept of a build lifecycle. This means that the process for building and distributing a particular artefact (or a whole project) is clearly defined. Developers only need a small set of commands to build any Gradle project, and the build.gradle will ensure they get the desired results.

There are three phases in the build lifecycle for Gradle, those being: Initialization, Configuration and Execution.

Initialization involves the identification of the settings file and evaluating which projects to include. Configuration evaluates the build scripts of each project to build a task dependency graph. Execution then takes this graph and executes the tasks in order of their dependencies. Some of the core tasks that you will be using within SENG202 include:

- **Run:** This will be the primary way of running your JavaFX applications, it is provided within the JavaFX plugin
- **Test:** Run the project's unit tests (and make sure they all pass) using the JUnit Framework.
- **Cucumber:** Runs Acceptance tests using the Given-When-Then format.
- **Jar:** Creates a Jar with all the dependencies of the project included within the Jar.
- **Clean:** Removes the previous build's artifacts such as the Jar and compiled classes.
- **Check:** Run the projects combined test suites.

It's important to note that each of these tasks have their own dependency trees and, in some cases, they will overlap each other. By having test as an independent task from run you will, in future, be able to add this task to your CI/CD pipeline. You can also customize tasks and add your own behaviour to Gradle tasks within the build.gradle. As an example, your team could create a task to have your Javadoc be generated when you run and application.

4 Dependency Management

Virtually all software projects have dependencies; usually imported libraries. Dependency management can be a huge problem for software engineers, particularly when working with large and complex projects. Manual dependency management involves:

- Downloading libraries (i.e., finding them on the internet);
- Installing libraries into the project directory;
- Periodically updating those libraries (which requires repeating the previous two steps);
- Telling co-workers which libraries to download, being sure to use the right version of a library for the project, using the right version of one library with the right version of another library, and;
- Making sure that all developers are using the same version of the same libraries for the same projects.

Hopefully you can see how things can start to go wrong. Gradle lets you list your project's dependencies (and their version numbers) using a predefined format. Listing 1 shows an example for the Joda Time library:

```
dependencies {
  implementation 'joda-time:joda-time:2.14.2'
}
```

Listing 1: Joda Time library dependency block

You can use Gradle to "resolve" your project's dependencies by downloading any versions of the libraries specified for the project which your computer may be missing. For example, if your computer had version 0.9.1 of the Joda Time library installed but your project's Gradle settings indicated that it needed version 2.14.2 Gradle would download the newer version and you would end up with two versions of the same library (the same would apply if you had version 2.14.2 installed but the project required version 0.9.1). This makes it much easier to set a project up on a new computer. Rather than having to work through a list of libraries and download them all individually, you can simply run Gradle, and it will do the work for you.

This creates an element of **repeatability**, which is one of Gradle's greatest strengths. By formally specifying and automatically executing the build process Gradle can reliably produce identical executable programs on different

computers at different times and under different conditions which comes with many benefits. Teams of developers can share their Gradle configuration files so that if one person adds code that requires a certain library the others don't have to worry about locating, downloading, and installing the correct version of that library. Gradle can also be useful when companies need to supply different customers with different versions of the same product, and for repeatedly testing a product with a continuous integration system like GitLab CI (or Jenkins), as you'll see later in the course.

Most popular IDEs offer support for Gradle, allowing you to do even less work. Although you do still need to understand how everything works, because one day it **will** fall apart around you.

4.1 Dependency Scopes

When adding dependencies to a Gradle project, the scope tells Gradle when that library should be available. The most common scopes are:

- **implementation**: used by your main application code and typically the default choice for most project dependencies.
- **testImplementation**: available only to your test code, so it is not included in the main application runtime.
- **runtimeOnly**: not needed when compiling the code, but required when the application is run.
- **compileOnly**: needed to compile the code, but not required at runtime.

Understanding these scopes helps keep your build organised and avoids accidentally making dependencies available in the wrong part of your project.

5 Setting up a Project with Gradle

5.1 Installing Gradle

Make sure you have Gradle installed on your development machine (or machines). It should already be installed in the university lab computers. The simplest way to do this is to type '`gradle –version`' into the command line. If you get a "command not found" error, please, make one of your tutors aware of this as soon as possible. For your home computer, you might need to download and install Gradle. The process for doing this varies with operating system. We recommend searching for an online tutorial suited to your environment. It's also a good idea to try and use the same version of Gradle as the rest of your team, as this reduces the number of inconsistencies between your environments (Lab computers use 9.2 whereas the latest version is 9.6).

6 Creating a Gradle Project

Now we'll use Gradle to create a "scaffold" for your new project. This just involves automatically creating a bunch of folders and some configuration files. You could do this manually, but it would take a long time and be prone to mistakes.

In the command line navigate to a suitable directory (this is where you'll create your project directory - you can always move it later, of course). Write '**gradle init**' into the shell and edit it as indicated. From here you will be prompted to choose a programming language, default package name etc. Feel free to experiment and see how things change but for this tutorial we suggest the following responses (for Gradle 9.2):

check this is correct with lab version

- *Project type* -> **Application**
- *Implementation Language* -> **Java**
- *Split Functionality* -> **No**
- *Build Script* -> **Groovy**
- *Generate build using new APIs* -> **No**
- *Test Framework* -> **JUnit Jupiter**
- *Project Name* -> **GradleExampleProject**

- *Source Package* -> **seng202.team[Your Team number]**
- *Target Version Of Java* -> **25**

Notes:

- The order and default values for many of these settings have changed in more recent versions. An example is the build script has shifted to Kotlin.

The directory structure that Gradle uses may look complex at first but it's intuitive once you see it laid out. It's designed to scale with large and complex projects, but it'll also work well for your needs. Open the new directory created by Gradle in a file browser and have a look around. The '**app/src**' folder is for your source code and contains sub-directories for your test and production code. The '**app/build.gradle**' file is particularly important, as that's where you configure Gradle.

In many projects you will see the '**app**' directory omitted and have '**src**' sit directly within the top level. This will likely be the case for the repository we provide for your project.

7 Introducing the Build.Gradle file

The **build.gradle** file is the core of a project's configuration in Gradle. It contains information about the project and configuration details used by Gradle to build the project. In the previous step you executed the Gradle initialization command with a set of parameters to create a simple Gradle project and the resulting '**build.gradle**' output can be seen in the Listing 2 below.

The **build.gradle** file can become large and its complexity can seem daunting, but it is not necessary to understand all the intricacies to use it effectively. However, it is important to understand the formatting of the file.

- **Plugins:** Software artifacts that are used to aid to Gradle to create the build artifacts for your project
- **Repositories:** Places that are available to gradle to find dependencies
- **Dependencies:** External Library builds that are used within the software application
- You'll note that the plugin '**application**' also has its own block at the bottom. Within Gradle it is possible to add configurations to plugins and dependencies in this way. Below we see that we set the main class for Application to know where to start the application from.

```
/*
 * This file was generated by the Gradle 'init' task.
 *
 * This generated file contains a sample Java application project to get you started.
 * For more details on building Java & JVM projects, please refer to https://docs.gradle.org/8.5/userguide/
      building_java_projects.html in the Gradle documentation.
 */

plugins {
    // Apply the application plugin to add support for building a CLI application in Java.
    id 'application'
}

repositories {
    // Use Maven Central for resolving dependencies.
    mavenCentral()
}

dependencies {
    // Use JUnit Jupiter for testing.
    testImplementation libs.junit.jupiter

    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'

    // This dependency is used by the application.
    implementation libs.guava
}

```

```
// Apply a specific Java toolchain to ease working on different environments.
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    // Define the main class for the application.
    mainClass = 'seng202.team0.App'
}

tasks.named('test') {
    // Use JUnit Platform for unit tests.
    useJUnitPlatform()
}
```

Listing 2: Gradle Project Imported into IntelliJ IDE

8 Importing a Gradle Project into your IDE

Throughout this course tutors will only provide support for the IntelliJ IDE, so unless you are comfortable with working in your IDE of choice, we suggest using IntelliJ [1].

IntelliJ

IntelliJ has built-in support for Gradle. To import your project:

- Launch IntelliJ and choose '**Open**' from the start up window (You may need to close your current project if you currently have one open).
- Select the project you created with Gradle in the previous section of this tutorial.
- Select okay and load in the project. You may be prompted if you trust the project, in this case select yes.
- Navigate through your project folder and open the build.gradle file, this should look something like Listing 2.

9 Setting up Java version compatibility

During the '**gradle init**' you were likely prompted to select a Java version. Within this course we will be using Java 25 as it is the version installed on the Lab Computers. You can always check the version of the machine you are on by typing '`java --version`' into a terminal. Make sure that when you are working from home you are using a version of Java that is compatible with the project.

If you're using IntelliJ and you have problems when you try to run your application, try updating the Java version to 25 in the following places:

- File -> Project structure -> Project -> Project SDK
- File -> Project structure -> Project -> Project language level
- File -> Project structure -> Project -> Modules -> Sources -> Language level
- File -> Settings -> Build, Execution, Deployment -> Compiler -> Java Compiler -> Target byte code version

10 Verifying the Project Scaffold

Now that you've imported your new Gradle project into your IDE you should make sure you can run it.

[1] You can use the professional version for free as a student by signing up for a JetBrains Educational License. See `https://www.jetbrains.com/community/education/#students`

10.1 IntelliJ

Locate '**App**' within the Project Window `src/main/java/seng202/team[your_team_number]/App`, right click on the file name and select:
Run/Run 'App.main()'
The default App.java file Gradle created when it scaffolded the project will print "Hello World!" as shown below, so make sure you see that output before proceeding.

Figure 1: Result of App.java being ran

```
6:01:01 pm: Executing ':app:App.main()'...

Starting Gradle Daemon...
Gradle Daemon started in 1 s 662 ms
Path for java installation '/usr/lib/jvm/openjdk-21' (Common Linux Locations) does not contain a java executable
Path for java installation '/usr/lib/jvm/openjdk-17' (Common Linux Locations) does not contain a java executable
Path for java installation '/usr/lib/jvm/openjdk-11' (Common Linux Locations) does not contain a java executable
> Task :app:compileJava
> Task :app:processResources NO-SOURCE
> Task :app:classes

> Task :app:App.main()
Hello World!

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

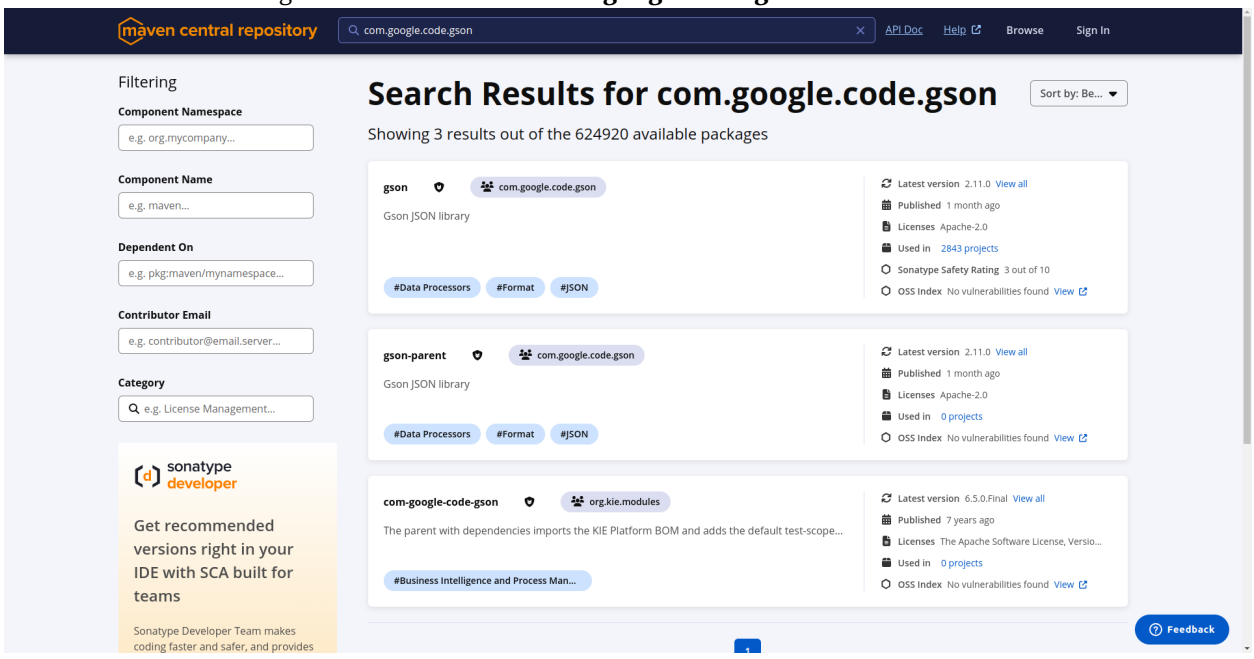
For more on this, please refer to https://docs.gradle.org/8.5/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentat

BUILD SUCCESSFUL in 15s
2 actionable tasks: 2 executed
6:01:19 pm: Execution finished ':app:App.main()'.
```

10.2 Searching for a Dependency

Now imagine that we want to use the popular Gson library (this is used to convert between Java and JSON objects). Rather than manually downloading the Gson .jar file and copying it into our project directory we'll add it to Gradle as a dependency. **Maven Central** (http://search.maven.org) is a repository containing thousands of libraries that we can easily import and use in our projects with Gradle allowing for much more efficient development. Search Maven Central for "**com.google.code.gson**" and select(*click on the version number to select it*) the most recent version as shown on Figure 2. At the date of this tutorial's release, the latest version was 2.14.0.

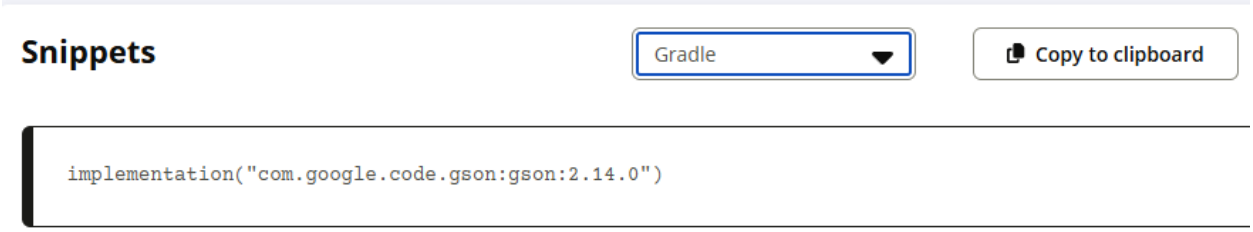
Figure 2: Maven Central: '**com.google.code.gson**' search outcome



10.3 Adding a Dependency

On this page you will see a card called Snippets. By default, it will be set to Apache Maven but by using the drop down you can also select Gradle or Gradle (short) (Figure 3). Both will work in any Gradle project, but we recommend following the standard that your team has discussed and set within your own coding standard.

Figure 3: Maven Central: '**com.google.code.gson**' Gradle(short) dependency block



Copy/paste this from the website into your project's **build.gradle** file (it should go inside the `dependencies{}` block, as shown in Listing 3 below). We also recommend commenting the use case and/or a link for explaining the dependency above so you can easily get more info about each dependency.

```
dependencies {
    // Use JUnit Jupiter for testing.
    testImplementation libs.junit.jupiter

    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'

    // This dependency is used by the application.
    implementation libs.guava

    implementation 'com.google.code.gson:gson:2.13.1'
}
```

Listing 3: **build.gradle** file with the 'google gson' dependency block

Your IDE should now detect that you've made changes to the **build.gradle** file and update the project accordingly (in our case, that just means resolving the newly added dependency). After this you should be able to use **Gson** in your project.

10.4 Verifying the New Dependency

To verify if the added dependency is working fine, let's create a dummy class named **Book** (Listing 4) which we will use to test the Gson library and refactor the **App** class (Listing 5) class to perform the test.

```
package seng202.team0;

public class Book {

    private String name;

    private String description;

    public Book(String name, String description) {
        this.name = name;
        this.description = description;
    }
}
```

Listing 4: Book class

```java
/*
 * This Java source file was generated by the Gradle 'init' task.
 */
package seng202.team0;

import com.google.gson.Gson;

public class App {

    public static void main(String[] args) {
        Book myBook = new Book("The Boy Who Morged", "A retelling of Harry Potter");

        Gson gson = new Gson();

        System.out.println("My Book: " + gson.toJson(myBook));

    }
}
```

Listing 5: App class

Run the program and check if you got the same output is as shown

Figure 4: Expected output for App.java

```
> Task :app:compileJava
> Task :app:processResources NO-SOURCE
> Task :app:classes

> Task :app:App.main()
My Book: {"name":"The Boy Who Morged","description":"A retelling of Harry Potter"}
```

Make sure everything worked fine before moving to the next sections, if you have any issues reach out to your tutors for support.

10.5 Version Management

If Gson (or another of your project's dependencies) was updated and you wanted to use a feature from the new version, all you would need to do is to edit the version number for that dependency in your project's **build.gradle** file. Your IDE would notice the change and ask Gradle to resolve the new version requirement, i.e. to reload the dependencies.

10.6 Sharing Gradle Configurations

In a team project, your **build.gradle** file would be added to your Git repository (or equivalent). Any changes made to the **build.gradle** by one member of the team would thus be shared with all other members.

10.7 Manually Adding Dependencies

It is possible to add dependencies which are not available on Maven Central, but we do not encourage doing so for SENG202. Manually adding jar files to your project is usually more trouble than it's worth. If you really want to do this, then please talk to one of your tutors so that together, you can find the best possible solution for the case.

11 Creating an Executable Jar

In the previous section (Dependency Management with Gradle) we looked at how useful Gradle is for handling dependency management. That's great for collaborating with other engineers, but what about when we deploy our product for customers to use? We don't want them to have to install Gradle, much less have all sorts of third-party jars downloaded to their system. It's better to give customers (and markers) a single runnable file (usually.jar) which contains all our source code and dependencies. Fortunately, there's a Gradle plugin which will do this for us!

Paste the `jar{}` block from Listing 6 into your **build.gradle** file and edit it to include your team number where indicated. By default, the jar command doesn't include dependencies within your jar file. This is an issue that will result in many classNotFound errors when trying to run the Jar. The Manifest block is responsible for resolving the main class of your application (Where your main function is) whilst the from block is responsible for packaging all of your applications dependencies within the Jar as well. This is what makes it a '**FatJar**'.

```
jar {
    duplicatesStrategy = DuplicatesStrategy.EXCLUDE

    manifest {
        attributes 'Main-Class': 'seng202.team0.App'
    }
    from {
        configurations.runtimeClasspath.collect {it.isDirectory()?it:zipTree(it)}
    }
}
```

Listing 6: Jar configuration added to the **build.gradle** file.

11.1 Packaging the Project and its Dependencies

In the command line navigate to your project directory and type '`./gradlew jar`'. The build process should start and successfully build your project as shown on Figure 5. As a result, Gradle should have created an executable jar at `[your_project_name]/app/build/libs/app.jar`. Run it by navigating to its folder in the command line and entering the command '`java -jar app.jar`'. You should see the text "Hello World!" in the shell (Figure 6).

Figure 5: '**./gradlew jar**' outcome, build success

```
[apa224@cs20117ks ~/gradlelab0]$ ./gradlew jar
Path for java installation '/usr/lib/jvm/openjdk-17' (Common Linux Locations) does not contain a java executable
Path for java installation '/usr/lib/jvm/openjdk-11' (Common Linux Locations) does not contain a java executable
Path for java installation '/usr/lib/jvm/openjdk-21' (Common Linux Locations) does not contain a java executable

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
```

Figure 6: Runnable file result

```
[apa224@cs20117ks ~/gradlelab0/app/build/libs]$ java -jar app.jar
My Book: {"name":"The Boy Who Morged","description":"A retelling of Harry Potter"}
```

Summarizing what has just happened, the '`./gradlew jar`' command told Gradle to complete the "jar" task. To do this Gradle must also execute any dependent tasks which precede it in the Task graph (see the list at the start of this tutorial).

If someone asks you to build a project with Gradle this is probably what they mean. They want you to use Gradle to produce an executable package with './gradlew jar' or a similar command.

Note that IntelliJ has a built-in interface for interacting with Gradle (View >Tool Windows > Gradle). You can use this if you would prefer to avoid the command line